

Developer Portfolio

Md. Jahid Hasan Raihan — Software Engineer

DOCUMENT VERSION

1.2

STATUS

Built & verified · Not yet deployed

STACK

Django 5 · DRF · React 18 · Vite · PostgreSQL

DATE

12 September 2026

OWNER

Md. Jahid Hasan Raihan

REPOSITORY

github.com/jhraihan

Purpose of this document. A complete reference for the portfolio project — who it represents, why each decision was made, how the system is built, and what remains outstanding. It is written so that a contributor with no prior context (including the author returning after a long gap) can pick the work up without re-deriving anything.

1. **Product Overview** — problem, goals, success criteria, non-goals
2. **Subject Profile** — identity, education, journey, skills, career direction
3. **Content Authenticity Rules** — the binding constraints on all content
4. **Information Architecture** — sitemap, page specifications
5. **Project Portfolio** — all five case studies in detail
6. **Design System** — palette, typography, spacing, motion
7. **Technical Architecture** — stack, data model, API, frontend structure
8. **Feature Specifications** — behaviour of each implemented feature
9. **Content Management** — how the owner edits the site
10. **Quality Requirements** — accessibility, performance, SEO, security
11. **Testing** — coverage and what is asserted
12. **Deployment** — target platforms and procedure
13. **Outstanding Work** — what is missing and who owns it
14. **Decision Log** — significant choices and their rationale
15. **Appendix** — file map, commands, credentials policy
16. **Revision Notes** — what each revision corrected, and why

01 Product Overview

1.1 Problem statement

A final-year Computer Science student with five substantial, self-built applications and no professional employment history needs a way to be evaluated on engineering ability rather than on résumé length. A conventional CV compresses five complete systems into five bullet points and discards everything that makes them evidence: the architecture, the trade-offs, and the problems that were genuinely hard to solve.

The portfolio exists to close that gap. It is the artefact a recruiter or engineering manager reads instead of, or alongside, the CV.

1.2 Target audience

AUDIENCE	WHAT THEY NEED	HOW THE SITE SERVES IT
Technical recruiters	Fast confirmation of stack, level, and location fit	Hero states name, title, stack and availability above the fold; project cards are scannable
Engineering managers	Evidence of real capability and judgement	Six-section case studies with architecture diagrams and an explicit "hardest problem" per project
Senior engineers (interview panel)	Something concrete to ask about	Request-flow diagrams, stated trade-offs, honest skill levels that hold up under questioning

1.3 Goals

#	GOAL	SUCCESS CRITERION
G1	A reader understands who the subject is within ten seconds	Name, title, core stack and availability all visible without scrolling
G2	Projects read as engineering, not as a gallery	Every project has problem, approach, architecture, hardest problem and lessons
G3	Nothing on the site can be contradicted in an interview	Every claim traces to something the subject stated or built
G4	Content is editable without touching code	All copy, links, media and skills are managed through Django admin
G5	The site itself demonstrates competence	Built in the stack it advertises; fast, accessible, tested
G6	Missing assets never look like defects	Empty fields hide their sections entirely; no "coming soon" states

1.4 Explicit non-goals

- **Not a blog.** Rejected — the owner is not committed to writing regularly, and a stale blog is worse than none.
- **Not an AI showcase.** No AI/ML skills section, no chatbot assistant. The owner is at the learning stage and declined to claim otherwise.
- **Not a design showpiece.** Motion supports hierarchy and never precedes content. No particle fields, cursor trails, or entry animations.
- **Not a CV replacement.** A résumé PDF is offered as a download; the site does not attempt to replicate it.

1.5 Positioning statement

ONE-LINE POSITIONING

A software engineer who builds complete, production-shaped web applications — not features in isolation.

Supported by five systems that each carry authentication, role-based access control, a relational schema, and a decoupled React client talking to a REST API. The repetition is the argument: this is a consistent competency, not a single lucky project.

02 Subject Profile

Everything in this section was supplied directly by the subject during a structured discovery interview. It is the only permitted source for content on the site.

2.1 Identity

FIELD	VALUE
Full name	Md. Jahid Hasan Raihan
Display name	Md. Jahid Hasan Raihan (full form, in hero and browser tab)
Professional title	Software Engineer
Location	Dhaka, Bangladesh — displayed publicly
Public email	jahidhr05@gmail.com — see 09
Availability	Open to remote, hybrid and onsite roles
Geography	Bangladesh and international
GitHub	github.com/jhraihan
LinkedIn	linkedin.com/in/md-jahid-hasan-raihan-745023393
LeetCode	leetcode.com/u/Jahid_Hasan_Raihan/ — ~150 problems, count not displayed

2.2 Education

FIELD	VALUE
Degree	BSc in Computer Science and Engineering
Institution	Independent University, Bangladesh
Duration	2022 → December 2026 (expected)
Standing	Final year
CGPA	3.30 — stored privately, not displayed (show_result = False)
Coursework shown	Data Structures · Algorithms · DBMS · Operating Systems · Computer Networks · OOP · Software Engineering
Thesis	None
Prior education	HSC/SSC deliberately omitted

2.3 Development journey

Programming began with the CSE degree in 2022 — not before it. The subject chose Computer Science out of genuine interest in the field, and coding became the part that held. Most knowledge beyond coursework is self-taught, primarily through online courses and iterative building.

Python → DSA & Algorithms → Django → Django REST Framework → Databases (MySQL, SQLite, PostgreSQL) → React

Django was chosen because Python already felt natural, and because it enabled building complete applications rather than fragments. Data structures and algorithms remain the practice ground for problem-solving; ~150 LeetCode problems solved to date.

2.4 Technical skills

Skills are classified by named level, never by percentage. A numeric bar implies a precision that does not exist and invites a comparison that cannot be won. Levels are ordered: Core → Strong → Working Knowledge → Familiar → Currently Learning → Exploring .

CATEGORY	CORE	STRONG	WORKING	FAMILIAR
Languages	Python	SQL, HTML, CSS	JavaScript	Java
Backend	Django, Django REST Framework, Django ORM	REST API design, Authentication (JWT / session / OAuth)	—	FastAPI, Node.js
Frontend	—	React, React Router, Fetch API	Tailwind CSS, Context API, Vite, Bootstrap	Next.js
Databases	—	MySQL, PostgreSQL, SQLite, Database design & normalisation	Query optimisation	—
DevOps	—	Git, GitHub, Environment management	Linux, Vercel / Netlify, Railway / Render / PythonAnywhere	Nginx, GitHub Actions, AWS
Tools	—	VS Code, Postman, Git CLI, AI coding tools	pgAdmin / MySQL Workbench	Figma

Featured five (surfaced most prominently): Django · Django REST Framework · React · MySQL · Python.

2.5 Experience and achievements

DELIBERATE ABSENCE

The subject has **no professional experience** — no jobs, internships, freelance or client work — and **no certifications, competitions or awards** to display. Both facts were confirmed explicitly.

The site therefore has **no Experience section and no Achievements section**. Empty sections were rejected as worse than absent ones, and fabricated content was ruled out entirely. The portfolio carries its weight through projects and demonstrated technical depth instead — which is both honest and normal for a final-year student.

2.6 Engineering philosophy

Stated priorities, in the subject's own ordering: **clean code · performance · correct architecture**.

Long-term direction: to become an **AI-native software engineer** — building software where modern AI is a first-class part of the system rather than an afterthought. This is recorded as a goal and, at the subject's instruction, is **not stated on the site**.

2.7 Career target

DIMENSION	TARGET
Roles	Junior Software Engineer · Backend Developer · Full-Stack Developer · Django Developer · Internship
Geography	Bangladesh and international
Company type	No preference — startup, established, or agency
Work mode	No preference — remote, hybrid or onsite
Industry	No preference
Portfolio PRD — Md. Jahid Hasan Raihan Availability	Graduating December 2026

2.8 Brand requirements

Intended ten-second impression: "This is a serious developer who can build real-world full-stack applications and is constantly pushing himself to learn and improve."

Intended lasting memory: "A hands-on full-stack developer who builds practical projects with Django and React, understands how the pieces fit together, and has the curiosity and persistence to keep getting better."

Tone: technical · professional · confident · friendly.

03 Content Authenticity Rules

These constraints override every other consideration, including making the subject look more impressive. They were agreed explicitly and are binding on all future contributors.

3.1 Prohibited content

NEVER INVENT

Jobs · companies · internships · clients · awards · certifications · user counts · revenue · statistics · years of experience · project features · technologies · achievements · testimonials.

Where information is missing, mark it [CONTENT NEEDED] and ask the owner. **Authenticity outranks impressiveness.**

3.2 Standing exclusions

EXCLUDED	REASON
AI / ML skills section	Owner is at the learning stage. No RAG, embeddings, vector databases, LangChain, agents or fine-tuning may appear as skills.
"Currently learning AI" line	Explicitly removed at the owner's request, after being drafted.
AI portfolio assistant	Proposed and declined.
Experience section	No professional history exists.
Achievements section	Nothing significant to display.
CGPA	Stored but withheld from the API unless <code>show_result</code> is enabled.
LeetCode count	Footer link only; the number invites comparison.
Skill percentages	Named levels only — see §2.4.

3.3 Permitted AI framing

Two projects (IntelliChat, PromptCanvas) integrate external model APIs. They are described **factually, as projects** — what they do and how they work. This is a description of shipped software, not a claim of AI expertise, and the distinction must be preserved.

3.4 Design failure modes to avoid

The owner explicitly asked the site to guard against all five:

FAILURE MODE	HOW IT MANIFESTS	COUNTERMEASURE APPLIED
Arrogance	"Expert", "master", superlatives	Factual copy only; no superlatives anywhere
"Just a student"	Apologetic language, "aspiring", timid type	Presented as a software engineer; degree is a credential, not a qualifier. <i>Highest-priority guard.</i>
Generic template	Purple gradients, skill bars, "Hi, I'm 🐙"	Custom design system; architecture diagrams; named skill levels
Over-designed	Particles, cursor trails, entry animations	Motion limited to fades, reveals and hover states
Unfinished	Broken links, "coming soon", placeholder text	Empty fields hide their section entirely; no dead buttons

4.1 Sitemap

Six routes are defined in `frontend/src/App.jsx` — five real pages plus the catch-all 404. With five project slugs, that resolves to **10 reachable pages** and **9 indexable URLs**.

```
/                Home
/projects        Project index, filterable by technology
/projects/<slug> Case study          x 5
/about          About, education, full skills
/contact        Contact form and direct links
*              404
```

4.2 Home page composition

#	SECTION	PURPOSE AND CONTENT
1	Hero	Availability pill, name at display scale, title, rotating typewriter line, intro paragraph, primary actions, photo with three factual stat cards
2	Featured projects	All five project cards in a responsive grid
3	Systems, not screens	Four engineering principles, each traceable to shipped code
4	What I work with	Six skill categories with level legend, plus university coursework
5	About preview	Condensed bio and education card
6	Contact CTA	Closing call to action with direct email

Hero typewriter content

Cycles five phrases, each factually true of the projects below:

```
REST APIs with Django & DRF
role-based access control
relational database schemas
JWT authentication flows
decoupled React frontends
```

Hero statistics

Three counters, all verifiable and none invented:

VALUE	LABEL	BASIS
5	full applications	Count of completed projects
4	databases used	MySQL, PostgreSQL, SQLite, Redis
150+	LeetCode problems	Owner-reported

4.3 Case study structure

Each of the five case studies renders up to six numbered sections. Numbering counts only sections that actually render, so a missing section never leaves a gap in the sequence.

#	SECTION	CONTENT
01	The problem	What the system solves and why it was built
02	The approach	Architecture chosen and why
03	Features	Complete feature list, two columns
04	Architecture	Request-flow diagram plus prose
05	Hardest problem	The genuinely difficult problem and its resolution
06	What I learned	Transferable lesson

Followed conditionally by: video walkthrough (if a URL exists) and screenshot gallery (if images exist). A persistent sidebar shows the stack.

05 Project Portfolio

Five projects, all solo, all complete, all with public repositories. Display order is fixed and set by the owner.

#	PROJECT	TYPE	FEATURES	CORE STACK
1	MicroMart	E-commerce platform	15	React · Django · DRF · MySQL · JWT · SSLCommerz
2	EduFlow	Learning management system	8	React · Vite · Tailwind · DRF · JWT
3	IntelliChat	Streaming AI chat	5	React · Django · PostgreSQL · Redis · SSE · Gemini
4	MediDesk	Hospital management	7	React · Django · DRF · JWT
5	PromptCanvas	Text-to-image generation	3	React · Django · PostgreSQL · Hugging Face

THE PATTERN

Every project is a role-based, multi-actor system with authentication, a relational database, and a decoupled React/DRF architecture. That repetition — not any single project — is the portfolio's central claim.

1. MicroMart

E-COMMERCE

Multi-role e-commerce platform with integrated payments.

React SPA → Django + DRF (JWT verified) → MySQL · branch: SSLCommerz

PROBLEM

A working e-commerce system is several overlapping applications. Customer, seller and administrator each need a different view of the same data with different permissions over it.

APPROACH

Fully decoupled: React SPA holding no server state, Django + DRF API owning all of it, MySQL underneath. JWT identifies which of three user types is making each request. Payments via SSLCommerz.

HARDEST PROBLEM

The complete order and payment workflow. Taking a payment is simple in isolation; keeping order, inventory and payment state consistent is not. Users abandon payments mid-flow, gateway callbacks arrive late or twice, and inventory must not be decremented for an unpaid order. Defining the states and their legal transitions was the bulk of the work.

LESSON

The interesting part of a feature is usually its failure cases. Once order states were explicit, most conditional logic scattered through the views disappeared.

FEATURES (15)

Registration & authentication · product search · voice search · shopping cart · payment processing · product reviews · order tracking · store management · product & inventory management · sales analytics · pricing management · user management · complaint management · fraud monitoring · seller verification

REPOSITORY

github.com/jhraihan/MicroMart

Role-based learning management system.

React + Vite (role-aware UI) → DRF (permission layer) → Database

PROBLEM

A learning platform is defined by who may do what. Teacher, student and administrator act on the same records with very different rights.

APPROACH

Role-based access control across three roles, enforced at the API and reflected in the interface. JWT authentication with lookup by phone number. Client state deliberately handled by native React hooks with no external state library.

HARDEST PROBLEM

Keeping one set of permission rules authoritative. Every endpoint must answer two questions, not one: may this user perform this action, and on which records. Answering only the first leaves a teacher able to reach another teacher's course by changing an id in the URL. The resolution: the interface never decides access — the API scopes every queryset by role, and the frontend hides controls purely as a convenience.

LESSON

Roles designed in from the start stay in one place; roles added afterwards become scattered conditionals. Building the frontend with only `useState`, `useEffect` and `Context` established firsthand where that stops being sufficient — for this size of application, it never did.

FEATURES (8)

Role-based access control · phone-based JWT authentication · course & lesson management · assignments & submissions · enrolment management · results · server-driven notifications · confirmation dialogs

REPOSITORY

github.com/jhraihaan/Learning-Management-System-django-react

3. IntelliChat

AI APPLICATION

Streaming AI chat application with persistent conversations.

React (SSE listener) → Django (stream relay) → PostgreSQL · branch: Gemini API

PROBLEM

A chat interface that waits for a complete model response before showing anything feels broken even when working. The response must arrive progressively and survive a page reload — both requirements change how the backend is built.

APPROACH

Django streams the model response over Server-Sent Events rather than returning a completed reply. Conversations and messages persist in PostgreSQL. Responses render as Markdown with syntax highlighting and a copy-code control.

ARCHITECTURE DETAIL

React posts to `/api/chat/`. Django calls Gemini and streams back a sequence: a `meta` event, repeated `delta` events carrying generated text, then a terminating `done` or `error` event.

HARDEST PROBLEM

Streaming abandons the familiar request/response shape — the connection stays open while text arrives in fragments. A stream can also end in more than one way: complete, fail midway, or the client disconnects during generation. The explicit event protocol exists so the frontend always knows which occurred, rather than inferring it from a stream that simply stopped.

LESSON

A persistence layer changes what an interface can promise: once conversations were stored rather than held in memory, a reload stopped being a data-loss event. Designing the client/server protocol first made both sides straightforward.

FEATURES (5)

Token-by-token streaming over SSE · persistent conversations · Markdown rendering · syntax highlighting with copy-code · authentication surviving reload

REPOSITORY

github.com/jhraihaan/llm_powered_ai_chatbot

Hospital management system with four-role access control.

React (4 role views) → Django + DRF (JWT + RBAC) → Database

PROBLEM

Hospital workflows are a permissions problem before they are a software problem. A receptionist books appointments but must not read clinical notes; a doctor issues prescriptions but does not manage billing; a patient sees only their own records.

APPROACH

JWT authentication with module access restricted by role across four roles. Appointments, prescriptions, records, billing and a medicine catalogue each expose only permitted operations.

HARDEST PROBLEM

Modelling a prescription properly. Storing it as text would have been quick, but a prescription is genuinely structured: one consultation links to several medicines, each with its own dosage and duration. Four roles also made access control harder than the three-role projects — each module had to be reasoned about separately rather than governed by one blanket rule.

LESSON

The schema decides what the application can do later. Because prescriptions were modelled as real relations, catalogue search and dosage linking came almost for free; a text field would have required a migration and a rewrite. An appointment is also a lifecycle, not a row — booked, approved, completed and cancelled are states with rules about legal transitions.

FEATURES (7)

Four-role JWT authentication · appointment management with filtering · structured prescriptions · doctor & patient portals · real-time availability toggle · billing & invoicing · searchable medicine inventory

REPOSITORY

github.com/jhraihaan/hospital-management-system-django-react

Text-to-image generation with stored results.

React (prompt input) → Django (server-side call) → PostgreSQL · branch: Hugging Face

PROBLEM

Built to work through what it takes to put a generative model behind a real interface: handling a request that takes seconds rather than milliseconds, keeping the model call server-side, and storing what comes back.

HARDEST PROBLEM

Latency breaks the usual assumptions — the interface must communicate that work is happening, and the backend holds a request open far longer than a typical API call. Keeping the model call server-side was the other constraint: the browser never contacts the inference API directly, so the credential stays on the server.

LESSON

What it takes to put an external model behind an interface rather than call it from a script. Storing generated images rather than discarding them turned a one-shot tool into something with history.

FEATURES (3)

Text-to-image generation via Hugging Face · image storage and retrieval · authentication required

REPOSITORY

github.com/jhraihaan/tech-store-platform-django-react

KNOWN ISSUE

The repository URL reads `tech-store-platform-django-react`, which does not match an image-generation project. Likely a renamed repository or a transcription error. **Must be verified before launch.**

6.1 Direction

Modern SaaS with minimal technical accents. The reference supplied by the owner was a dark agency site with a red accent; its visual language was adopted, its content architecture deliberately was not — that site leads with client counts and testimonials, none of which exist here.

6.2 Colour

Defined once as CSS custom properties in `src/styles/index.css` and consumed everywhere as semantic Tailwind tokens. Nothing hardcodes a colour, so theme switching never requires touching a component.

TOKEN	DARK (DEFAULT)	LIGHT	ROLE
--canvas	8 8 10	251 251 252	Page background
--surface	15 16 19	255 255 255	Cards, raised areas
--elevated	23 24 28	244 245 247	Inputs, tags, nested surfaces
--line	38 39 45	226 227 232	Default border
--line-strong	56 57 65	203 205 213	Hover / emphasis border
--ink	250 250 251	13 14 18	Primary text
--ink-muted	158 160 170	82 84 94	Body text
--ink-faint	108 110 121	124 127 138	Metadata, labels
--accent	226 58 50	194 40 33	Single accent, used sparingly

The accent is deliberately deeper than the reference's brighter red: it keeps the energy while reading as more controlled, and passes AA contrast on dark backgrounds where the brighter tone struggles at small sizes. Light mode deepens it further for contrast on white.

Four secondary hues — cyan, amber, violet, green — exist solely for syntax-style category tinting in skill groups and architecture diagram nodes. They are never used for interface state.

6.3 Typography

ROLE	FAMILY	USAGE
Sans	Inter	All reading text, headings, interface labels
Mono	JetBrains Mono	Metadata, eyebrows, tags, stack lists, diagram nodes, keyboard hints

The scale clamps so headings never overflow small screens or grow absurd on wide ones:

TOKEN	SIZE	USAGE
text-hero	clamp(3rem, 7.5vw, 6.5rem)	Hero name only
text-display	clamp(2.25rem, 4.5vw, 3.5rem)	Page titles
text-heading-lg	clamp(1.75rem, 3vw, 2.5rem)	Section headings
text-heading	clamp(1.375rem, 2vw, 1.75rem)	Subsection headings
text-body	1.0625rem / 1.7	Prose

NAMING TRAP — DO NOT REPEAT

The hero size was originally named `display-xl`. Tailwind failed to generate it because it collides with the existing `display` key, silently falling back to 16px. It was renamed to `hero`. **Avoid font-size keys that prefix-collide with existing ones.** Note also that Tailwind config changes require a Vite restart — they are not hot-reloaded.

6.4 Motion

EFFECT	BEHAVIOUR
Scroll reveal	Fade and 16px rise as elements enter the viewport
Typewriter	Hero line cycles five phrases with a blinking caret
Magnetic buttons	Primary actions drift toward the cursor; transform applied directly to the node, costing no re-renders
Spotlight cards	Radial glow follows the cursor; position written to CSS custom properties, not React state
Ambient glows	Two slow drifting blurs behind the hero
Scroll progress	Hairline accent bar under the header, appearing once scrolled
Skeletons	Shimmer placeholders matching real content shape, preventing layout shift

NON-NEGOTIABLE MOTION RULE

Content must never depend on an animation to become visible. The `Reveal` component enforces this two ways: reduced-motion renders a plain element with no animation, and a 900ms timer reveals the element regardless — so a missed intersection (a collapsed grid child, an offscreen container, a browser without `IntersectionObserver`) can never leave real content stuck at zero opacity.

This rule exists because it was violated once: case study sections rendered into the DOM but stayed permanently invisible, because `whileInView` never fired inside a collapsed grid. Every section of every case study was affected.

7.1 Stack

LAYER	TECHNOLOGY	RATIONALE
Frontend	React 18 · Vite · Tailwind CSS · Framer Motion · React Router · Lucide	The stack the site advertises; Vite for fast builds
Backend	Django 5 · Django REST Framework	Owner's core competency
Database	PostgreSQL (production) · SQLite (local default)	Postgres is the best-supported and cheapest-to-host option on managed platforms; SQLite lets the project run before credentials exist
Testing	Django test runner · Vitest · Testing Library	Native to each side
Config	python-decouple · .env	No secrets in source

7.2 Repository layout

```
portfolio_part4/
├── backend/
│   ├── apps/
│   │   ├── profiles/      # singleton owner profile
│   │   ├── projects/      # projects, technologies, features, images
│   │   ├── skills/        # skill categories, skills, coursework
│   │   ├── education/     # degrees
│   │   └── contact/       # inbound messages
│   ├── config/            # settings, urls, api_urls
│   ├── requirements/      # base / dev / prod
│   └── manage.py
├── frontend/
│   └── src/
│       ├── components/    # reusable UI
│       ├── sections/      # home page sections
│       ├── pages/         # routed pages
│       ├── layouts/       # header, footer
│       ├── hooks/         # useApi, useTheme, useTypewriter, ...
│       ├── services/      # api client
│       └── styles/         # theme tokens
├── images/                # original source images (git-ignored)
├── README.md
└── DEPLOYMENT.md
```

7.3 Data model

MODEL	APP	NOTES
Profile	profiles	Singleton — enforced in <code>save()</code> and in the admin. Holds identity, hero copy, about text, photo, résumé, social links, SEO description.
Project	projects	Identity, classification, five optional case study fields, three optional URLs, cover image, ordering, featured flag.
ProjectFeature	projects	Ordered feature list, FK to Project.
ProjectImage	projects	Screenshot gallery with caption and alt text; alt falls back to caption, then to a generated description.
Technology	projects	Shared M2M tag with a category. Drives the project filter.
SkillCategory / Skill	skills	Named levels with an ordering weight, so the strongest surface first.
CourseworkSubject	skills	Studied subjects, listed without any proficiency claim.
Education	education	Result stored separately from <code>show_result</code> , so a CGPA can be recorded privately.
ContactMessage	contact	Stored rather than emailed, so nothing is lost if SMTP is unconfigured. IP and user agent captured for abuse triage only.

7.4 API surface

The public API is **read-only**. The single write endpoint is the contact form, which opts into `AllowAny` and its own throttle scope.

METHOD	ENDPOINT	RETURNS
GET	<code>/api/profile/</code>	The singleton profile; empty object if none exists
GET	<code>/api/projects/</code>	Project list; accepts <code>?tech=<slug></code> and <code>?featured=true</code>
GET	<code>/api/projects/<slug>/</code>	Full case study with features and images
GET	<code>/api/technologies/</code>	Technologies attached to at least one project
GET	<code>/api/skills/</code>	Categories with nested skills, strongest first
GET	<code>/api/coursework/</code>	Studied subjects
GET	<code>/api/education/</code>	Degrees; <code>result</code> is <code>null</code> unless published
POST	<code>/api/contact/</code>	Accepts a message; throttled 5/hour per IP

7.5 Notable frontend patterns

PATTERN	DETAIL
<code>useApi</code>	Tracks loading, data and error. An incrementing request id means a response arriving after unmount, or after a newer request, never sets state.
Shared fetches	Profile and project list are fetched once in <code>App</code> and passed down — both are needed by several pages and the command palette, and neither changes during a session.
Stretched link	Project cards use one focusable link with an <code>::after</code> overlay, keeping a single tab stop. Icon links sit above it with <code>z-10</code> .
Conditional rendering	Every optional URL, image and case study section renders only when populated. This is the mechanism behind the "no unfinished states" rule.
Theme	Applied before first paint by an inline script in <code>index.html</code> , preventing a flash of the wrong theme. All storage access is wrapped in try/catch.

08 Feature Specifications

8.1 Architecture diagrams

The highest-value feature. Each case study renders a request-flow diagram as inline SVG — no image request, crisp at any zoom, inheriting theme tokens. Flow definitions live in `frontend/src/components/ArchitectureDiagram.jsx`; a project with no defined flow renders nothing rather than an empty frame.

Each diagram is framed in terminal-window chrome with a caption, uses colour-coded nodes, and optionally hangs an external service off the API node as a dashed branch. IntelliChat's connectors animate to signify streaming. On narrow screens the flow stacks vertically with downward connectors, so no node is ever clipped.

Rationale: this shows systems thinking rather than claiming it, and gives an interviewer something concrete to ask about.

8.2 Command palette

Opened with `Cmd/Ctrl + K` from anywhere. Groups results into Navigation, Case studies, Actions and Links. Actions are derived from live data, so a newly added project appears without code changes. Full keyboard control: arrows navigate, Enter selects, Escape closes — Escape is bound at the document so it works regardless of focus.

8.3 Contact form

CONCERN	HANDLING
Fields	Name, email, subject, message — all required, all validated server-side
Validation feedback	Per-field errors with <code>aria-invalid</code> and <code>aria-describedby</code> ; an error clears as soon as its field is edited
Honeypot	A hidden <code>website</code> field. Anything that fills it is rejected with a generic message that reveals nothing.
Spam patterns	Regex checks for common spam markers and three-or-more links
Rate limiting	5 submissions per hour per IP, via a DRF scoped throttle
Storage	Written to the database and read in the Django admin — never exposed through the API
Email	Optional. If SMTP is configured a notification is sent, but delivery failure never fails the request, since the message is already stored.

8.4 Media placeholders

`ImageFrame` renders a designed placeholder when no image exists: a dotted field with corner ticks, holding the correct aspect ratio. It reads as part of the design rather than as an error, which is what allows the site to ship before screenshots exist. Supplying an image later requires no other change. The component also falls back to the placeholder if an image fails to load.

8.5 Project filtering

The active filter lives in the URL query string, so a filtered view is shareable and survives refresh and back-navigation. Only nine interview-relevant technologies are offered as filters — listing all 34 turns the control into a wall of buttons nobody reads.

09 Content Management

Everything visible on the site is editable through the Django admin at `/admin/`. No content requires a code change.

TO EDIT	GO TO
Name, title, hero copy, about text, links, photo, résumé	Profiles → Profile
Projects, case studies, features, screenshots	Projects → Projects
Technology tags	Projects → Technologies
Skills and levels	Skills → Skill categories
Coursework	Skills → Coursework subjects
Degree	Education → Education
Inbound messages	Contact → Contact messages

9.1 Adding the outstanding assets

ASSET	PROCEDURE
Screenshots	Open a project → <i>Project images</i> → upload, set order and alt text. The placeholder is replaced automatically.
Video walkthrough	Paste a YouTube URL into the project's <code>video_url</code> . An embedded player appears in the case study.
Live demo	Paste the URL into <code>live_url</code> . The button appears; while empty, no dead button is shown.
Résumé	Upload a PDF to Profile → <code>resume</code> . Download buttons appear in the hero and about page.
Repository URL	Edit <code>github_url</code> on the project.

After adding or renaming a project, regenerate the sitemap:

```
python manage.py generate_sitemap --base-url https://yourdomain.com
```

10 Quality Requirements

10.1 Accessibility

All verified programmatically against the running site:

- Semantic HTML with a `main` landmark
- Skip link as the first tab stop
- Exactly one `h1` per page
- `lang` set on the document
- Every image carries alt text
- Every button and link has an accessible name
- Visible focus ring for keyboard users only
- Form errors linked via `aria-describedby`
- Filter buttons expose `aria-pressed`
- Result regions marked `aria-live="polite"`
- All motion disabled under `prefers-reduced-motion`

10.2 Performance

MEASURE	RESULT
Total bundle, gzipped	≈124 KB
Vendor split	React, Framer Motion and app code in separate chunks
Profile photo	3.1 MB source → 96 KB optimised (800×800, progressive JPEG)
Images	Lazy by default; hero and first cards eager
Loading states	Shape-matched skeletons rather than spinners, preventing layout shift
Fonts	Preconnected, <code>display=swap</code>

10.3 SEO

- Per-page title and meta description, set on navigation
- Open Graph and Twitter Card tags
- JSON-LD `Person` schema with job title, location, institution and verified profile links
- Generated `sitemap.xml` covering **9 URLs** — 4 static paths plus one per project — alongside `robots.txt`. The 404 route is correctly excluded.
- Canonical URL — **currently `example.com` ; must be updated at launch**

10.4 Security

CONTROL	IMPLEMENTATION
Secrets	<code>.env</code> , git-ignored; <code>.env.example</code> documents every variable. Verified absent from the repository.
API exposure	Read-only except the contact endpoint
Contact abuse	Honeypot, spam patterns, 5/hour throttle
Private data	CGPA withheld unless explicitly published; message IP and user agent never serialised
Production headers	HSTS, SSL redirect, secure cookies, nosniff, <code>X-Frame-Options: DENY</code> — all active when <code>DEBUG=False</code>
Verification	<code>manage.py check --deploy</code> passes clean

11 Testing

55 automated tests, all passing — 35 backend, 20 frontend. Per-suite counts below were re-derived from the source files on 12 September 2026 and sum to those totals.

SUITE	TESTS	COVERS
Backend — contact	11	Valid submission, email normalisation, every validation failure, honeypot, spam patterns, metadata capture, read-blocking, throttle firing on the sixth request
Backend — projects	12	Slug generation, ordering, featured and technology filters, detail payload, 404, write-rejection, unused technologies excluded
Backend — education	4	CGPA withheld by default, returned once published, absent from the raw response body , ongoing date range
Backend — profiles	5	Singleton enforcement, updates still permitted, empty-state response, read-only
Backend — skills	3	Level ordering, human-readable display, no numeric proficiency exposed
Backend total	35	
Frontend — ImageFrame	3	Image renders, placeholder substitutes, loading attribute
Frontend — ProjectCard	6	Content, case study link, GitHub link, live demo hidden without a URL , technology collapsing
Frontend — ArchitectureDiagram	5	Known flows, external branch, branch omitted when absent, nothing for unknown slug
Frontend — CommandPalette	6	Closed state, listing, filtering, empty result, Escape, links omitted when absent
Frontend total	20	

Several tests encode the authenticity rules directly — CGPA absence from the raw body, no numeric proficiency fields, no dead demo buttons — so a future change that breaks a promise made in §03 fails the suite rather than shipping.

Manual verification also covered: every page in both themes with zero console errors — the five real routes, all five case studies, and the 404 state — plus mobile overflow on every page, reduced-motion behaviour, and the contact form end to end in a real browser.

12 Deployment

Status: not yet deployed. Full procedure in `DEPLOYMENT.md`.

COMPONENT	TARGET	NOTES
Frontend	Vercel	Free; needs <code>vercel.json</code> rewrite so client routes do not 404 on refresh
Backend + PostgreSQL	Railway	≈\$5/month; no cold starts. A single <code>DATABASE_URL</code> configures the database.
Domain	Namecheap / Cloudflare	≈\$12/year — reads far better than a platform subdomain

Render was explicitly rejected: its free tier sleeps after 15 minutes, producing a ~50 second cold start that reads as a broken site.

KNOWN DEPLOYMENT CONSTRAINT

Railway's filesystem is ephemeral, so uploaded screenshots are lost on redeploy. Once images are added regularly, move media to object storage (S3 or Cloudflare R2, which has a free tier) via `django-storages`.

12.1 Launch checklist

- `.env` absent from git
- Fresh `SECRET_KEY` generated for production
- `DEBUG=False`
- `ALLOWED_HOSTS` set to the real domain
- CORS and CSRF origins set to the real frontend URL
- Admin password changed from the development default
- `seed_portfolio` run
- Canonical URL and OG tags updated from `example.com`
- Sitemap regenerated with the real domain
- Contact form submits successfully in production
- Refreshing a case study URL does not 404
- All five repository links verified

13 Outstanding Work

The site is complete and shippable. Every item below is wired to hide rather than break while absent, and all are content changes through the admin — none require code.

#	ITEM	OWNER	NOTES
01	Screenshots for all five projects	Subject	Designed placeholders in place. Highest visual impact of the remaining work.
02	YouTube walkthroughs	Subject	Chosen in place of deploying. Embed appears once a URL is set.
03	Résumé PDF	Subject	Download buttons hidden until uploaded.
04	Verify repository URLs	Subject	All five are being reorganised. PromptCanvas URL is a known mismatch.
05	Deployment	Subject	Procedure documented; not yet executed.
06	Canonical URL and OG tags	Either	Still <code>example.com</code> . Must change at launch.
07	Provision PostgreSQL	Either	Currently SQLite locally. Set <code>USE_SQLITE=False</code> plus either <code>DATABASE_URL</code> or the <code>DB_*</code> values, then migrate and seed.
08	OG preview image	Either	<code>/og-image.png</code> is referenced but does not exist. Link previews are currently imageless.
09	Confirm the public email	Subject	The site publishes <code>jahidhr05@gmail.com</code> . A second address, <code>hello.jahidhr@gmail.com</code> , is also in active use. Confirm which belongs on the site; if it changes, edit it in <i>Profile</i> and in <code>seed_portfolio.py</code> so a reseed does not revert it.

13.1 Optional future enhancements

Considered and deliberately deferred, not rejected:

- **Email notifications** — the code path exists; only SMTP variables are needed.
- **Object storage for media** — required once screenshots are added and the site is on Railway.
- **Additional projects** — the model and command palette pick them up automatically; only an architecture flow definition would need adding.

14 Decision Log

#	DECISION	RATIONALE
D1	Named skill levels, not percentages	A bar implies precision that does not exist and invites unwinnable comparison. Enforced by a test.
D2	No Experience or Achievements section	Nothing genuine to show. An empty section is worse than an absent one; a padded one is disqualifying.
D3	No AI skills claimed	Owner is at the learning stage and asked for this explicitly, including removal of a drafted "currently learning" line.
D4	Full case studies for all five projects	Five substantial case studies read stronger than three plus two thin cards.
D5	Empty fields hide their section	The mechanism that lets the site ship before screenshots, demos and résumé exist without looking unfinished.
D6	Architecture diagrams as inline SVG	Shows systems thinking rather than claiming it. No network request; theme-aware; crisp at any zoom.
D7	Reference site's visuals adopted, its content architecture rejected	The reference is an agency selling services, leading with client counts and testimonials. None of those exist here, and inventing them was ruled out.
D8	Photo in the hero rather than only in About	Owner deferred the choice. The reference balanced its hero with a figure; a real photograph serves recruiters better than an illustration.
D9	Accent deepened from the reference red	Retains the energy, reads more controlled, and passes AA contrast where the brighter tone fails at small sizes.
D10	Command palette included	Small, genuinely useful, and an immediate technical signal to the intended audience.
D11	GitHub statistics excluded	Would surface repositories the owner is still reorganising.
D12	Blog excluded	Only worthwhile if maintained; a stale blog is worse than none.
D13	Contact messages stored, not emailed	Nothing is lost if SMTP is unconfigured. Notification is additive.
D14	SQLite default locally, PostgreSQL in production	Originally MySQL, switched on 12 Sep 2026: managed Postgres is cheaper and more widely available, <code>psycopg</code> installs without system build tools, and it is Django's best-tested path. SQLite still lets the project run before credentials exist.
D15	<code>Reveal</code> force-reveals after 900ms	Fixes a real defect where case study sections rendered but stayed invisible. Content must never depend on an animation.
D16	<code>DATABASE_URL</code> takes precedence over discrete settings	Every managed Postgres host issues one connection string. Supporting it means production needs a single database variable instead of five, and the same settings file serves local and hosted environments unchanged.

15.1 Commands

```
# Backend
cd backend
venv\Scripts\activate                # Windows
pip install -r requirements/dev.txt
python manage.py migrate
python manage.py seed_portfolio        # loads all real content, idempotent
python manage.py seed_portfolio --reset # wipe and reload
python manage.py createsuperuser
python manage.py runserver            # 127.0.0.1:8000, admin at /admin/

# Frontend
cd frontend
npm install
npm run dev                          # localhost:5173, /api proxied to Django
npm run build

# Tests
cd backend && python manage.py test    # 35
cd frontend && npm test                # 20

# Sitemap
python manage.py generate_sitemap --base-url https://yourdomain.com
```

15.2 Key files

PATH	CONTAINS
backend/apps/profiles/management/commands/seed_portfolio.py	All site content. Single source of truth for the seed.
frontend/src/styles/index.css	Theme tokens for both palettes
frontend/tailwind.config.js	Type scale, spacing, keyframes
frontend/src/components/ArchitectureDiagram.jsx	Per-project flow definitions
frontend/src/components/Reveal.jsx	Scroll reveal with the visibility guarantee
backend/config/settings.py	Environment-driven configuration
backend/.env.example	Every required variable, documented

15.3 Credentials policy

BEFORE ANY DEPLOYMENT

The development superuser (jahid) uses a weak local password that **must be changed before deployment**. Generate a fresh SECRET_KEY for production and never reuse the development one. .env is git-ignored and verified absent from the repository — keep it that way.

15.4 Development environment note

VS Code may report unresolved imports for django , rest_framework and decouple . This is the editor pointing at the system Python rather than the virtual environment. Fix with *Python: Select Interpreter* → backend\venv\Scripts\python.exe . The code is correct.

16 Revision Notes

Each revision re-verifies the document against the repository rather than against the previous revision. What changed, and why:

16.1 v1.1 — 12 September 2026

The database moved from **MySQL to PostgreSQL**. Managed Postgres is cheaper and far more widely available on free and low-cost tiers, `psycopg` installs without system build tools, and it is Django's best-tested path. Settings, requirements, `.env.example`, `DEPLOYMENT.md` and `README.md` were all updated together, and §7, §12, O7, D14 and D16 rewritten to match.

WHY MYSQL STILL APPEARS IN THIS DOCUMENT

MySQL is correctly retained in three places and must not be edited out: as a **skill** (§2.4 — the subject's strongest database, and one of the featured five), as **MicroMart's database** (§05), and in the **learning journey** (§2.3). Only the portfolio site's own database changed. Both sets of statements are true simultaneously.

16.2 v1.2 — 12 September 2026

A verification pass over every counted or enumerated claim, checked against the source files rather than carried forward. Five corrections:

#	PREVIOUSLY STATED	VERIFIED REALITY
R1	Backend contact suite: 10 tests	11 — counted in <code>apps/contact/tests.py</code>
R2	Backend skills suite: 4 tests	3 — counted in <code>apps/skills/tests.py</code>
R3	Per-suite rows did not sum to the stated 55, and no subtotals were shown, so the discrepancy was invisible	Backend and frontend subtotal rows added (35 and 20). The table is now self-checking.
R4	"Sitemap covering all nine routes"	Routes and URLs were conflated. 6 routes are defined; the sitemap holds 9 URLs (4 static + 5 case studies), and the 404 is correctly excluded.
R5	"Manual verification covered all ten routes"	Ten <i>pages</i> were checked, not ten routes. Restated without the false route count.

One new outstanding item was opened rather than silently resolved: **O9**, a discrepancy between the published email address and a second address in active use. The document records both and asks, per §3.1, rather than guessing which is correct.

16.3 Keeping this document current

Re-verify and re-render whenever any of the following changes: the database or hosting choice, the test counts, the routes, the five projects or their repository URLs, or any authenticity rule in §03. Sections **03** (authenticity), **13** (outstanding work) and **14** (decision log) go stale fastest. Counts in §11 should be re-derived from the test files, never copied from the previous revision.

Portfolio PRD v1.2 — Md. Jahid Hasan Raihan · 12 September 2026. Every factual claim in this document originates from the subject's own statements or from project state verified against the repository on the date above. Where information was unavailable it is marked outstanding in §13 rather than assumed. Companion document: `docs/ABOUT_JAHID.pdf`.